

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

***** 📄 *****



SOICT

BÁO CÁO BÀI TẬP LỚN HỌC PHẦN HỆ NHÚNG

(Mã học phần: IT4210)

Đề tài: Game Frog Jump

Giảng viên hướng dẫn: TS. Ngô Lam Trung

Mã lớp học: 168505

Sinh viên thực hiện: Nhóm Siêu thị điện máy H2C

Cao Chí Công - 20235281

Vương Toàn Minh Hiếu - 20235330

Nguyễn Hữu Hiệu - 20235331

Hà Nội, tháng 07 năm 2026

MỤC LỤC

MỤC LỤC.....	2
Chương I. Tổng quan đề tài & Phân công công việc.....	4
1.1. Giới thiệu đề tài.....	4
1.2. Các công nghệ sử dụng.....	4
1.3. Phân công công việc.....	4
Chương II. Thiết kế phần cứng hệ thống.....	5
2.1. Sơ đồ tổng quan hệ thống.....	5
2.2. Các linh kiện phần cứng.....	5
2.3. Cấu hình các ngoại vi.....	6
Bảng 3: Cấu hình các chân ngoại vi.....	8
2.3.1. Cấu hình cho ngoại vi nút bấm điều khiển nhân vật.....	8
2.3.2. Cấu hình ngoại vi mạch khuếch đại cho phát hiệu ứng âm thanh...	11
Chương III. Thiết kế phần mềm hệ thống.....	16
3.1. Thiết kế giao diện trên TouchGFX Designer.....	16
3.1.1. Screen1 - Màn hình khởi động (Start Screen).....	16
3.1.2. Screen2 - Màn hình chơi chính (Gameplay).....	16
3.1.3. Screen3 - Màn hình kết thúc (End Screen).....	17
3.1.4. Quản lý Text/Font.....	17
3.1.5. Chiến lược bộ đệm & màu sắc.....	17
3.2. Kiến trúc mã nguồn, mô hình MVP.....	18
3.2.1. Kiến trúc phân tầng & Quản lý ứng dụng.....	18
3.2.2. Mô hình Model-View-Presenter.....	18
3.3. Thuật toán Logic Game và xử lý sự kiện.....	20
3.3.1. Cấu trúc dữ liệu và vòng lặp game (handleTickEvent).....	20
3.3.2 Thuật toán sinh chương ngại vật giả ngẫu nhiên.....	21
Chương IV. Kết quả thực nghiệm & Đánh giá.....	22
4.1. Đánh giá độ trễ xử lý & Hiệu năng hệ thống.....	22
4.2. Đánh giá tài nguyên sử dụng bộ nhớ.....	22
Chương V. Kết luận & Hướng phát triển.....	24
5.1. Kết quả đạt được.....	24
5.2. Những hạn chế.....	24
5.2.1. Lỗi tích hợp và xung đột trong quá trình sinh mã nguồn tự động...	24
5.2.2. Hiện tượng dội phím (bouncing) đối với các nút bấm.....	25
5.2.3. Chưa tích hợp nhạc nền trò chơi.....	25
5.3. Hướng phát triển.....	26

5.4 Lời cảm ơn.....	26
PHỤ LỤC.....	28

Chương I. Tổng quan đề tài & Phân công công việc

1.1. Giới thiệu đề tài

Đề tài hướng tới xây dựng trò chơi “Frog Jump” trên kit STM32F429I Discovery, sử dụng TouchGFX để thiết kế giao diện đồ họa trực quan và xử lý chuyển động mượt mà. Cùng với đó, hệ thống tích hợp phát âm thanh trò chơi thông qua chuẩn giao tiếp I2S DMA nhằm nâng cao trải nghiệm người dùng.

1.2. Các công nghệ sử dụng

- Nền tảng phần cứng: STM32F429I Discovery Kit với là vi điều khiển trung tâm STM32F429ZIT6 (ARM Cortex-M4, 2MB Flash, 256KB RAM nội, 8MB SDRAM ngoại).
- Phần mềm: STM32CubeIDE (môi trường lập trình), TouchGFX Designer 4.26.1 (thiết kế đồ họa), STM32CubeMX (cấu hình phần cứng)

1.3. Phân công công việc

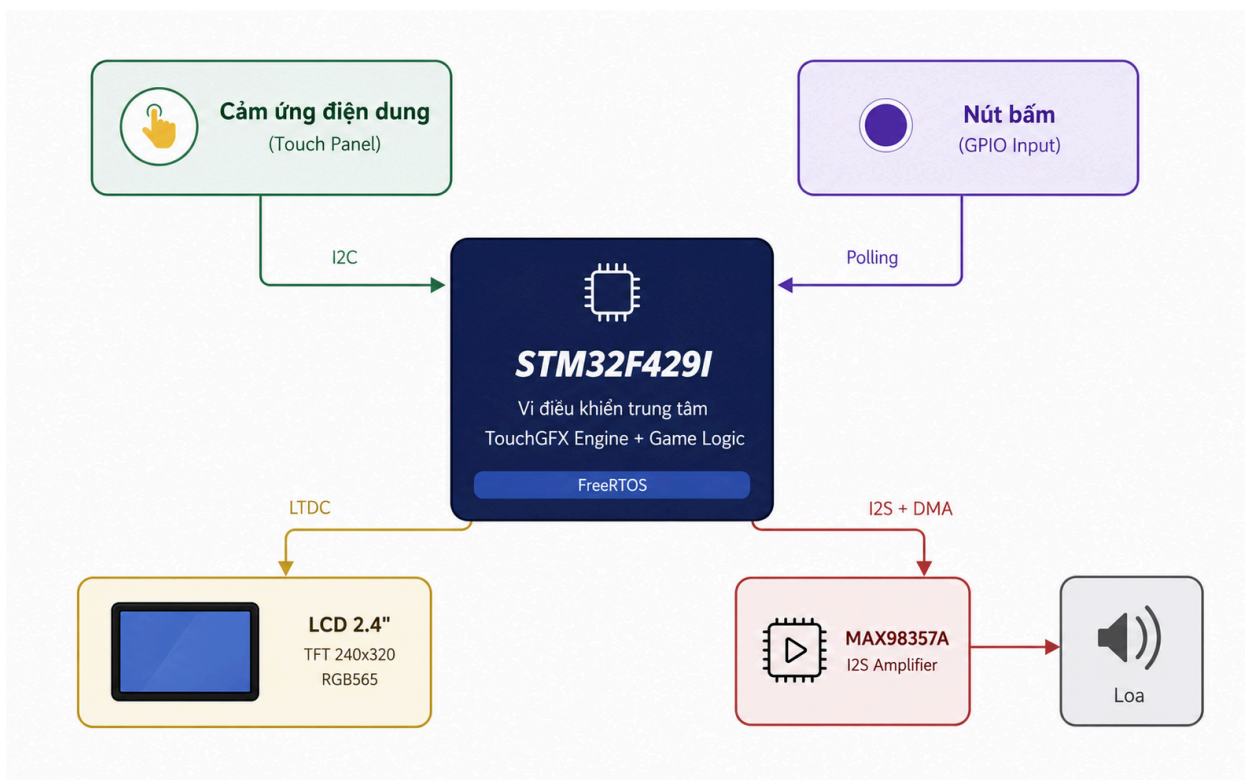
- **Các công việc chung:** Thiết kế tổng thể hệ thống, kiểm thử hệ thống, soạn báo cáo.
- **Các công việc riêng:**

Họ và tên	MSSV	Công việc
Cao Chí Công	20235281	Thiết kế giao diện UI (các màn hình, các đối tượng, các vật thể), xử lý chuyển màn hình.
Nguyễn Hữu Hiệu	20235331	Thiết kế logic game (điều khiển nhân vật, va chạm, tính điểm); cấu hình phần cứng GPIO (nút bấm) và I2S DMA (âm thanh).
Vương Toàn Minh Hiếu	20235330	Xử lý chuyển màn hình, thiết kế màn chơi (sinh ngẫu nhiên đối tượng vật cản, đường đi và background).

Bảng 1. Bảng phân chia công việc các thành viên trong nhóm

Chương II. Thiết kế phần cứng hệ thống

2.1. Sơ đồ tổng quan hệ thống



Hình 1. Sơ đồ tổng quan hệ thống

2.2. Các linh kiện phần cứng

Hệ thống được xây dựng trên board STM32F429I-DISCOVERY (rev D01) của STMicroelectronics, tích hợp sẵn các thành phần cần thiết cho một sản phẩm hiển thị đồ họa nhúng, nên đề tài không cần thiết kế PCB rời mà tập trung vào đầu nối ngoại vi và xử lý các sự kiện.

STT	Linh kiện	Thông số chính	Vai trò trong hệ thống
1	Vi điều khiển STM32F429ZIT 6	ARM Cortex-M4F, xung nhịp lõi 180 MHz, FPU phần cứng, 2 MB Flash, 192 KB RAM (SRAM) + 64 KB CCM RAM	Xử lý trung tâm: chạy FreeRTOS, engine đồ họa TouchGFX, logic game

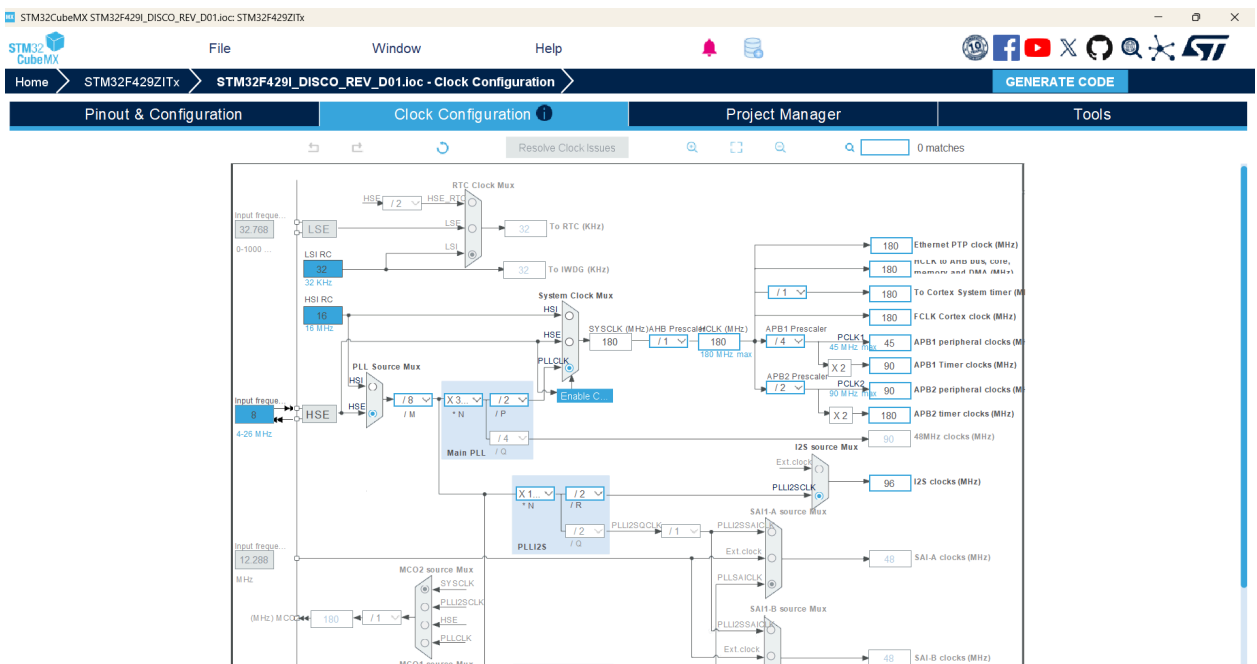
2	Bộ tạo dao động thạch anh ngoài	HSE 8 MHz (PH0/PH1)	Nguồn xung nhịp chính, tạo SYSCLK 180 MHz
3	SDRAM ngoài	8 MB, giao tiếp qua FMC (Flexible Memory Controller), bus dữ liệu 16-bit	Vùng nhớ mở rộng, dùng làm framebuffer & bộ đệm ảnh cho TouchGFX
4	Màn hình LCD TFT tích hợp	Độ phân giải 240×320 , định dạng màu RGB565 (16 bpp), giao tiếp qua LTDC (LCD-TFT Display Controller)	Hiển thị giao diện người dùng và trò chơi
5	Cảm ứng điện dung tích hợp	IC STMPE811 (giao tiếp I2C3)	Bắt các sự kiện chạm trên màn hình để thực hiện các chức năng phần mềm tương ứng
7	Nút bấm vuông (push button)	Nút bấm 4 chân, kích thước 12x12 mm	Nhận tín hiệu để điều khiển hướng di chuyển của nhân vật chính (mèo)
8	Mạch khuếch đại MAX98357A	Công suất tối đa 3W, điện áp hoạt động 2.5-5.5 V, giao tiếp qua giao thức I2S	DAC chuyển đổi tín hiệu số ra tín hiệu tương tự, khuếch đại tín hiệu để phát ra loa
9	Loa	Công suất 3W, điện trở kháng 8 Ohm	Phát hiệu ứng âm thanh trong quá trình chơi.

Bảng 2. Bảng thông tin linh kiện phần cứng

2.3. Cấu hình các ngoại vi

Trước khi cấu hình các chân trên board và kết nối ngoại vi, cần thiết lập hệ thống xung nhịp (RCC) để cung cấp nguồn xung ổn định cho vi điều khiển và các bus ngoại vi. Việc cấu hình xung nhịp phù hợp đảm bảo CPU và các ngoại vi

hoạt động đúng tần số thiết kế, đáp ứng yêu cầu về hiệu năng và độ ổn định. Hệ thống sử dụng nguồn xung HSE 8 MHz và bộ PLL (Phase-Locked Loop) để tạo xung nhịp hệ thống SYSCLK 180 MHz. Trong đó, AHB (Advanced High-performance Bus) hoạt động ở 180 MHz để kết nối lõi CPU với bộ nhớ và các ngoại vi tốc độ cao; APB1 (Advanced Peripheral Bus 1) hoạt động ở 45 MHz và APB2 (Advanced Peripheral Bus 2) hoạt động ở 90 MHz để cấp xung cho các ngoại vi. Cấu hình này đáp ứng yêu cầu xử lý đồ họa thời gian thực và đảm bảo các giao tiếp như SPI, I2C, I2S hoạt động ổn định.



Hình 2. Sơ đồ cấu hình Clock

Việc thiết lập các chân sử dụng cho các ngoại vi (ngoại vi không nằm trên board) được thực hiện thông qua STM32 CubeMX nhằm thuận tiện hóa quá trình thiết kế phần cứng, nhờ hỗ trợ cấu hình ngoại vi bằng giao diện đồ họa trực quan và tự động sinh mã khởi tạo (ghi tại file .ioc), giúp giảm thời gian phát triển cũng như hạn chế sai sót trong quá trình cấu hình. Chi tiết các chân cấu hình cho ngoại vi ngoài board được đề cập trong bảng dưới đây.

Chân MCU	Cấu hình	Chức năng
PG2	GPIO Input (Pull-up)	Di chuyển nhân vật sang phải
PG3	GPIO Input (Pull-up)	Di chuyển nhân vật lùi xuống
PA5	GPIO Input (Pull-up)	Di chuyển nhân vật tiến lên

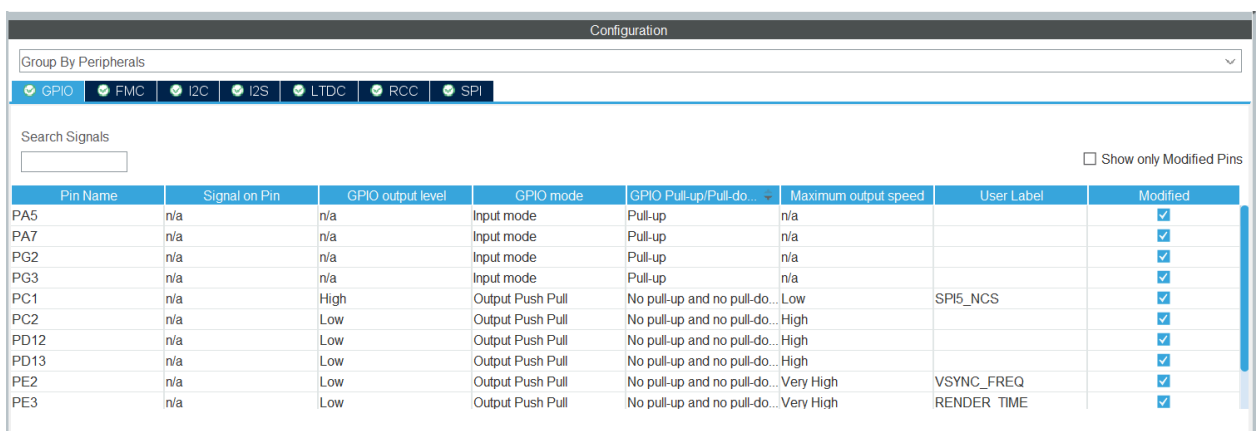
PA7	GPIO Input (Pull-up)	Di chuyển nhân vật sang trái
PC10	I2S3_CK	I2S Clock (đồng bộ clock giữa bên truyền và bên nhận)
PC12	I2S3_SD	I2S Data (truyền dữ liệu)
PA15	I2S3_WS	I2S LRC (xác định hướng âm thanh bên trái hoặc bên phải)

Bảng 3. Cấu hình các chân ngoại vi

2.3.1. Cấu hình cho ngoại vi nút bấm điều khiển nhân vật

a) Cấu hình phần cứng

Đề tài sử dụng 04 nút bấm để điều khiển hướng di chuyển của nhân vật trong trò chơi. Mỗi nút bấm được kết nối trực tiếp với một chân GPIO của vi điều khiển STM32, trong đó một đầu của nút được nối với chân GPIO, đầu còn lại được nối chung xuống đất (GND).



Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-down	Maximum output speed	User Label	Modified
PA5	n/a	n/a	Input mode	Pull-up	n/a		☒
PA7	n/a	n/a	Input mode	Pull-up	n/a		☒
PG2	n/a	n/a	Input mode	Pull-up	n/a		☒
PG3	n/a	n/a	Input mode	Pull-up	n/a		☒
PC1	n/a	High	Output Push Pull	No pull-up and no pull-do...	Low	SPI5_NCS	☒
PC2	n/a	Low	Output Push Pull	No pull-up and no pull-do...	High		☒
PD12	n/a	Low	Output Push Pull	No pull-up and no pull-do...	High		☒
PD13	n/a	Low	Output Push Pull	No pull-up and no pull-do...	High		☒
PE2	n/a	Low	Output Push Pull	No pull-up and no pull-do...	Very High	VSYNC_FREQ	☒
PE3	n/a	Low	Output Push Pull	No pull-up and no pull-do...	Very High	RENDER_TIME	☒

Hình 3. Cấu hình các chân kết nối

Các chân GPIO được cấu hình ở chế độ GPIO Input kết hợp với điện trở kéo lên nội (Pull-up), nhờ đó không cần nối thêm điện trở vào các nút. Với cấu hình này, khi nút bấm không được nhấn, điện trở Pull-up sẽ kéo chân GPIO lên mức điện áp 3.3 V, tương ứng với mức logic HIGH (GPIO_PIN_SET). Khi nhấn nút, chân GPIO được nối trực tiếp xuống GND nên điện áp tại chân giảm xuống 0 V, tương ứng với mức logic LOW (GPIO_PIN_RESET). Cấu hình Pull-up giúp chân GPIO luôn duy trì ở một mức logic xác định khi nút không được tác động,

tránh hiện tượng tín hiệu bị trôi nổi (floating) và tăng độ ổn định trong quá trình đọc trạng thái nút bấm.

b) Cơ chế hoạt động

Để phát hiện trạng thái của các nút bấm, hệ thống có thể sử dụng một trong hai cơ chế là ngắt ngoài (External Interrupt - EXTI) hoặc thăm dò (Polling).

- **Ngắt ngoài (EXTI):** Vi điều khiển chỉ thực hiện xử lý khi phát hiện có sự thay đổi mức logic tại chân GPIO (cạnh lên hoặc cạnh xuống). Khi đó, phần cứng sẽ kích hoạt một ngắt và chương trình sẽ chuyển đến hàm phục vụ ngắt (Interrupt Service Routine - ISR) để xử lý sự kiện nhấn nút. Cơ chế này có ưu điểm là phản hồi nhanh và không cần kiểm tra trạng thái nút liên tục.
- **Thăm dò (Polling):** Vi điều khiển định kỳ đọc trạng thái của các chân GPIO để xác định nút có được nhấn hay không. Cơ chế này có cách triển khai đơn giản, dễ quản lý và phù hợp với các ứng dụng trao đổi dữ liệu ngắn hoặc không yêu cầu phản hồi tức thời.

Trong hệ thống này, cơ chế Polling được lựa chọn do chỉ sử dụng bốn nút bấm điều khiển, dữ liệu gửi đi đơn giản chỉ là 1 bit 0 hoặc 1, không yêu cầu thời gian đáp ứng ở mức micro giây và việc kiểm tra trạng thái định kỳ hoàn toàn đáp ứng yêu cầu của trò chơi. Đồng thời, Polling giúp giảm độ phức tạp của chương trình khi không cần xây dựng các hàm phục vụ ngắt và xử lý đồng bộ giữa ngắt với các tác vụ khác trong hệ thống sử dụng FreeRTOS.

Việc đọc trạng thái các nút bấm được thực hiện trong một tác vụ (task) của FreeRTOS với chu kỳ 20 ms, sử dụng hàm `osDelay(20)`. Hệ điều hành FreeRTOS được cấu hình với tần số tick là 1000 Hz, tương ứng 1 tick là 1 ms, do đó `osDelay(20)` sẽ tạo khoảng trễ 20 ms, tương đương tần số lấy mẫu 50 Hz. Với chu kỳ này, hệ thống có thể phát hiện thao tác nhấn của người dùng một cách nhanh chóng, liên tục nhưng vẫn đảm bảo tính ổn định.

Tại mỗi chu kỳ, tác vụ sẽ gọi hàm `HAL_GPIO_ReadPin()` để đọc trạng thái của từng nút bấm. Nếu chân GPIO trả về `GPIO_PIN_RESET`, hệ thống xác định nút đang được nhấn; ngược lại, khi trả về `GPIO_PIN_SET`, nút được xác định là đang ở trạng thái không được nhấn. Khi phát hiện một nút được nhấn, tác vụ sẽ tạo lệnh điều khiển tương ứng với hướng di chuyển (lên, xuống, trái hoặc phải) và gửi lệnh này vào hàng đợi (Queue) của FreeRTOS bằng cơ chế truyền thông giữa các tác vụ. Tác vụ điều khiển nhân vật sẽ nhận lệnh từ hàng đợi và cập nhật

hướng di chuyển của nhân vật trong trò chơi. Việc sử dụng Queue giúp tách biệt tác vụ đọc nút bấm và tác vụ xử lý logic trò chơi, đồng thời đảm bảo việc truyền dữ liệu giữa các tác vụ diễn ra an toàn và đồng bộ.

```
uint32_t hold_pa7 = 0;
uint32_t hold_pa5 = 0;
uint32_t hold_pg3 = 0;
uint32_t hold_pg2 = 0;

/* Infinite loop */
for(;;)
{
    GPIO_PinState curr_pa7 = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7); /* LEFT */
    GPIO_PinState curr_pa5 = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5); /* UP */
    GPIO_PinState curr_pg3 = HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_3); /* DOWN */
    GPIO_PinState curr_pg2 = HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_2); /* RIGHT */

    /* Process PA7 (LEFT) */
    if (curr_pa7 == GPIO_PIN_RESET)
    {
        if (prev_pa7 == GPIO_PIN_SET)
        {
            uint16_t cmd = CMD_CAT_LEFT;
            osMessageQueuePut(myCommandQueueHandle, &cmd, 0, 0);
            hold_pa7 = 0;
        }
        else
        {
            hold_pa7++;
            if (hold_pa7 > 15)
            {
                uint16_t cmd = CMD_CAT_LEFT;
                osMessageQueuePut(myCommandQueueHandle, &cmd, 0, 0);
                hold_pa7 = 10;
            }
        }
    }
    else
    {
        hold_pa7 = 0;
    }
    prev_pa7 = curr_pa7;
}
```

Hình 4. Ví dụ bắt sự kiện nút bấm PA7 bằng polling

Ngoài ra, để nâng cao trải nghiệm điều khiển, hệ thống triển khai cơ chế bấm giữ liên tục (Auto-Repeat) cho các nút điều khiển, tương tự cơ chế hoạt động của bàn phím máy tính với hai giai đoạn: thời gian chờ trước khi lặp và tần số lặp.

Khi người dùng nhấn một nút lần đầu, hệ thống ngay lập tức gửi một lệnh điều khiển tương ứng vào hàng đợi (Queue) của FreeRTOS để nhân vật phản hồi tức thời. Đồng thời, một biến đếm được khởi tạo nhằm theo dõi thời gian giữ nút. Trong khoảng 300 ms đầu tiên khi giữ nút, hệ thống chỉ tăng giá trị của biến đếm mà không phát sinh thêm lệnh điều khiển. Khoảng thời gian chờ này giúp phân biệt thao tác nhấn một lần với giữ nút, tránh trường hợp người dùng chỉ muốn di chuyển một bước nhưng do giữ phím lâu hơn một chút mà nhân vật thực hiện nhiều bước liên tiếp.

Sau khi thời gian giữ vượt quá 300 ms, hệ thống chuyển sang chế độ nhấn giữ phím. Các lệnh điều khiển sẽ được gửi định kỳ vào hàng đợi với chu kỳ 100 ms, tương đương 10 lệnh/giây (10 Hz), giúp nhân vật di chuyển liên tục khi người dùng giữ nút. Để đạt được tần số lặp này, sau mỗi lần gửi lệnh, biến đếm không được đặt lại về giá trị ban đầu mà được đưa về một giá trị trung gian. Nhờ đó, số chu kỳ cần đếm trước lần lặp tiếp theo được rút ngắn, đảm bảo tốc độ lặp ổn định mà không cần sử dụng thêm bộ định thời (Timer) hay tác vụ riêng.

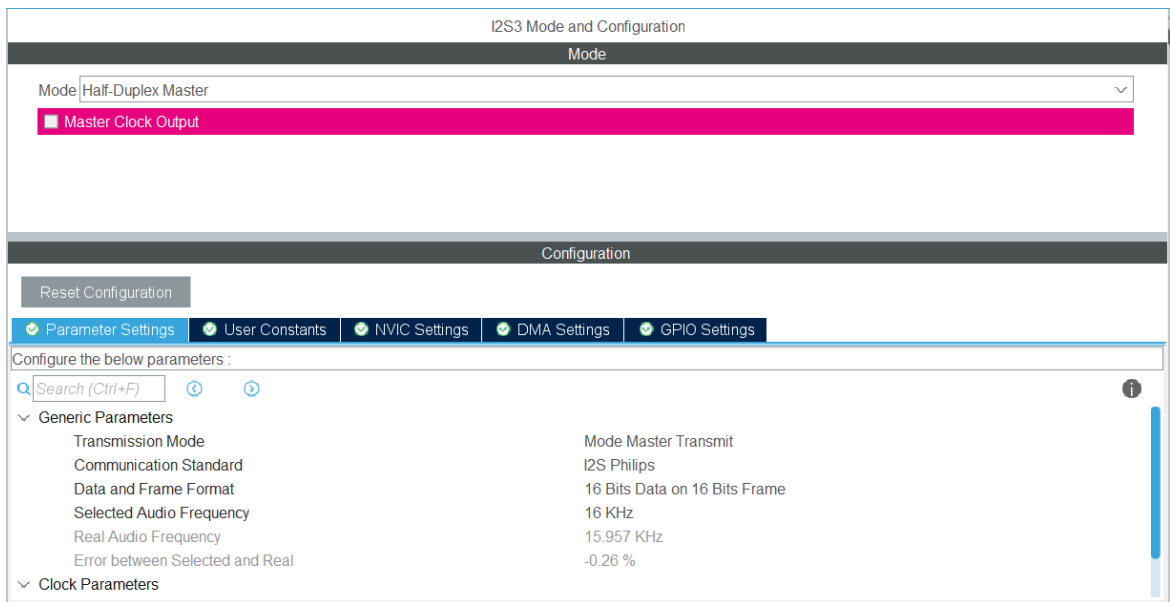
Khi người dùng nhả nút, biến đếm được đặt lại về không, chấm dứt cơ chế nhấn giữ và đưa hệ thống trở về trạng thái sẵn sàng cho lần nhấn tiếp theo. Với chu kỳ thăm dò nút bấm 20 ms, cơ chế này được cấu hình với thời gian chờ bắt đầu lặp là 300 ms và tần số lặp 10 Hz, đảm bảo nhân vật phản hồi nhanh khi nhấn lần đầu, đồng thời mang lại cảm giác điều khiển liên tục, tự nhiên và dễ kiểm soát khi giữ nút trong quá trình chơi trò chơi.

2.3.2. Cấu hình ngoại vi mạch khuếch đại cho phát hiệu ứng âm thanh

a) Cấu hình phần cứng

Để phát các hiệu ứng âm thanh trong trò chơi, hệ thống sử dụng mạch khuếch đại âm thanh số MAX98357A kết hợp với loa công suất 3W, trở kháng 8Ω. MAX98357A là mạch DAC tích hợp bộ khuếch đại Class-D, có nhiệm vụ nhận dữ liệu âm thanh số từ vi điều khiển thông qua giao tiếp I2S (Inter-IC Sound), chuyển đổi tín hiệu số sang tín hiệu tương tự và khuếch đại đủ công suất để điều khiển loa phát âm thanh.

Ngoại vi SPI3 của vi điều khiển được cấu hình hoạt động ở chế độ I2S3 với vai trò Master Transmitter (I2S_MODE_MASTER_TX), cho phép vi điều khiển chủ động tạo các tín hiệu đồng hồ và truyền dữ liệu âm thanh đến MAX98357A. Giao tiếp được cấu hình theo chuẩn Philips I2S, sử dụng định dạng dữ liệu PCM 16-bit với tần số lấy mẫu 16 kHz, phù hợp với yêu cầu phát các hiệu ứng âm thanh trong trò chơi. Do hệ thống chỉ thực hiện phát hiệu ứng âm thanh và không cần thu tín hiệu âm thanh từ bên ngoài, giao tiếp I2S được cấu hình ở chế độ Half-Duplex (Transmit Only), không sử dụng đường truyền dữ liệu nhận (RX). Đường Master Clock (MCLK) cũng không được sử dụng do MAX98357A chỉ yêu cầu ba tín hiệu chính gồm Bit Clock (BCLK), Word Select (WS/LRCK) và Serial Data (SD hay DIN) để đồng bộ và nhận dữ liệu âm thanh.



Hình 5. Thiết lập giao thức I2S cho truyền dữ liệu âm thanh

Để đảm bảo độ chính xác của tần số lấy mẫu mà không ảnh hưởng đến xung nhịp hệ thống, vi điều khiển sử dụng PLLI2S làm nguồn xung chuyên biệt cho ngoại vi I2S. Với cấu hình PLLI2SN bằng 192 và PLLI2SR bằng 2, PLLI2S tạo nguồn xung 96 MHz cho khối I2S. Từ nguồn xung này, bộ chia xung nội của ngoại vi I2S tự động tạo các tín hiệu Bit Clock (BCLK) và Word Select (WS) tương ứng với cấu hình PCM 16-bit, tần số lấy mẫu 16 kHz, đảm bảo dữ liệu âm thanh được truyền đúng tốc độ và đồng bộ với mạch MAX98357A.

I2S3 Mode and Configuration							
Configuration							
Reset Configuration							
Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings							
Search Signals							
Search (Ctrl+F) Show only Modified Pins							
Pin Name	Signal on Pin	GPIO ou.	GPIO mode	GPIO Pull-up/Pull-down	Maximum output sp.	User Label	Modified
PA15	I2S3_WS	n/a	Alternate Function Push Pull	No pull-up and no pull-down	Low		☐
PC10	I2S3_CK	n/a	Alternate Function Push Pull	No pull-up and no pull-down	Low		☐
PC12	I2S3_SD	n/a	Alternate Function Push Pull	No pull-up and no pull-down	Low		☐

Hình 6. Cấu hình các chân giao tiếp I2S trên vi điều khiển

Bằng việc thiết lập thông qua STM32 CubeMX, các chân tương ứng phục vụ cho giao tiếp I2S cũng được thiết lập. Cụ thể, chân PA15 được sử dụng làm I2S3_WS (Word Select/LRCK) để xác định khung dữ liệu âm thanh ứng với kênh bên trái hoặc kênh bên phải, chân PC10 được cấu hình làm I2S3_CK (Bit Clock/BCLK) để đồng bộ quá trình truyền từng bit dữ liệu, và chân PC12 được cấu hình làm I2S3_SD (Serial Data) để truyền luồng dữ liệu âm thanh số đến MAX98357A. Các chân đều được cấu hình ở chế độ Push-Pull, không sử dụng điện trở kéo lên, đáp ứng yêu cầu của giao tiếp I2S.

Sơ đồ kết nối phần cứng giữa vi điều khiển, mạch MAX98357A và loa được thể hiện trong Hình 2.x. Trong đó, vi điều khiển truyền dữ liệu âm thanh số thông qua giao tiếp I2S đến MAX98357A. Mạch này thực hiện chuyển đổi tín hiệu số sang tín hiệu tương tự, sau đó khuếch đại tín hiệu bằng bộ khuếch đại Class-D và xuất công suất trực tiếp đến loa 3 W để phát các hiệu ứng âm thanh của trò chơi.

b) Cơ chế hoạt động

Trong hệ thống, dữ liệu âm thanh được truyền từ bộ nhớ đến mạch khuếch đại MAX98357A thông qua giao tiếp I2S trước khi được giải mã và phát ra loa ngoài. Để nâng cao hiệu năng, quá trình truyền dữ liệu không được thực hiện trực tiếp bởi CPU mà sử dụng truy xuất bộ nhớ trực tiếp (Direct Memory Access - DMA).

DMA Request	Stream	Direction	Priority
SPI3_TX	DMA1 Stream 5	Memory To Peripheral	Low

Add
Delete

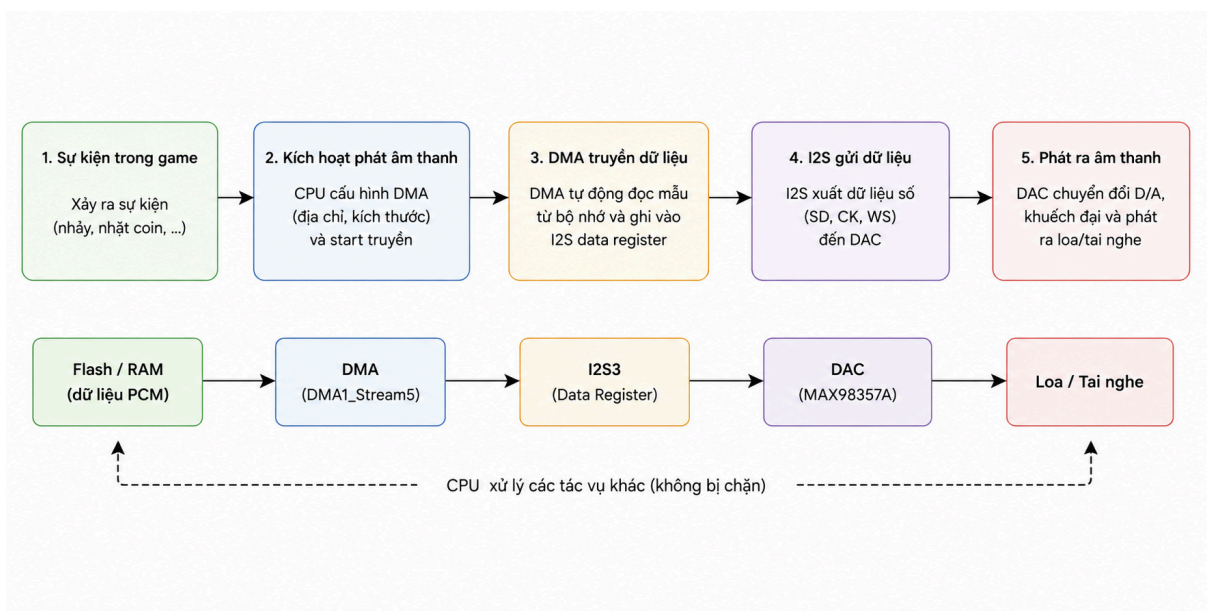
DMA Request Settings

Mode: Normal	Increment Address: ☐	Peripheral: ☐	Memory: ☒
Use Fifo: ☐	Threshold:	Data Width: Half Word	Half Word
		Burst Size:	

Hình 7. Thiết lập DMA cho giao tiếp I2S

Nếu CPU tự ghi từng mẫu dữ liệu âm thanh vào thanh ghi của ngoại vi I2S3, bộ xử lý sẽ phải liên tục phục vụ quá trình truyền trong suốt thời gian phát âm

thanh, làm giảm khả năng xử lý các tác vụ khác như cập nhật giao diện TouchGFX, xử lý nút bấm hoặc tính toán logic của trò chơi. Việc sử dụng DMA cho phép quá trình truyền dữ liệu được tự động ủy quyền cho DMA Controller và thực hiện hoàn toàn bằng phần cứng, giúp giảm tải cho CPU và nâng cao khả năng xử lý đồng thời của hệ thống.



Hình 8. Sơ đồ quá trình giao tiếp I2S sử dụng DMA

Khi có yêu cầu phát một hiệu ứng âm thanh, CPU chỉ thực hiện cấu hình bộ điều khiển DMA với địa chỉ vùng nhớ chứa dữ liệu âm thanh PCM 16-bit và số lượng mẫu cần truyền, sau đó kích hoạt quá trình truyền đến ngoại vi I2S3. Sau khi hoàn tất việc cấu hình, CPU không cần tham gia vào quá trình truyền dữ liệu nữa mà có thể tiếp tục thực hiện các tác vụ khác. Toàn bộ việc đọc dữ liệu từ bộ nhớ và ghi đến thanh ghi dữ liệu của ngoại vi I2S được DMA đảm nhiệm theo cơ chế bất đồng bộ (Asynchronous/Non-Blocking).

Trong quá trình hoạt động, mỗi khi bộ đệm truyền của I2S3 sẵn sàng nhận dữ liệu mới, ngoại vi sẽ tự động phát tín hiệu yêu cầu đến DMA. Khi nhận được yêu cầu, DMA đọc mẫu âm thanh 16-bit tiếp theo từ bộ nhớ và ghi trực tiếp vào thanh ghi dữ liệu của I2S3. Quá trình này được lặp lại liên tục cho đến khi toàn bộ dữ liệu của tệp âm thanh được truyền hết. Nhờ cơ chế này, dữ liệu luôn được cung cấp đúng thời điểm theo tốc độ lấy mẫu đã cấu hình mà không cần CPU liên tục can thiệp.

Sau khi nhận dữ liệu từ DMA, ngoại vi I2S3 chuyển đổi các mẫu PCM thành luồng dữ liệu nối tiếp và xuất ra ba tín hiệu Serial Data (SD), Bit Clock (BCLK)

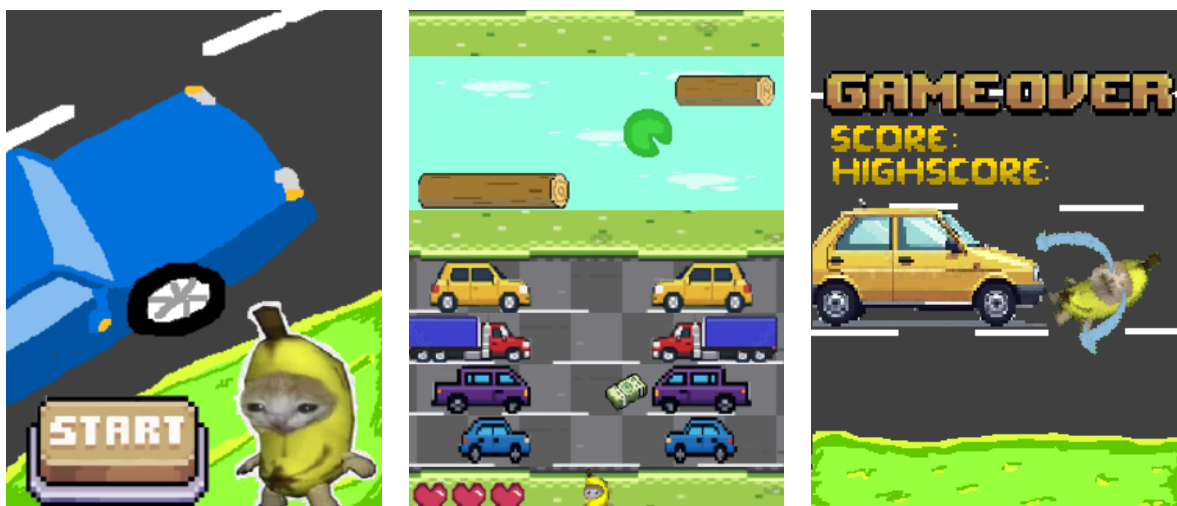
và Word Select (WS). Trong đó, tín hiệu BCLK đồng bộ quá trình truyền từng bit dữ liệu, WS xác định khung dữ liệu của từng mẫu âm thanh, còn SD mang dữ liệu PCM được truyền đến mạch khuếch đại MAX98357A. Dựa trên các tín hiệu này, MAX98357A thực hiện chuyển đổi tín hiệu âm thanh số sang tín hiệu tương tự bằng bộ DAC tích hợp, sau đó khuếch đại bằng mạch khuếch đại Class-D và xuất trực tiếp đến loa để phát các hiệu ứng âm thanh của trò chơi.

Khi DMA truyền hết dữ liệu của một hiệu ứng âm thanh, phần cứng sẽ tự động sinh ngắt để thông báo quá trình truyền đã hoàn tất. Ngoại vi I2S được đưa trở về trạng thái sẵn sàng và chờ yêu cầu phát âm thanh tiếp theo. Toàn bộ quá trình phát âm thanh được thực hiện theo cơ chế non-blocking, trong đó CPU chỉ tham gia ở giai đoạn khởi tạo truyền dữ liệu, còn việc truyền và đồng bộ dữ liệu được thực hiện hoàn toàn bởi DMA và ngoại vi I2S. Nhờ đó, hệ thống vẫn duy trì được khả năng xử lý giao diện, đọc nút bấm và thực hiện các tác vụ khác trong khi hiệu ứng âm thanh đang được phát.

Chương III. Thiết kế phần mềm hệ thống

3.1. Thiết kế giao diện trên TouchGFX Designer

Giao diện được thiết kế bằng TouchGFX Designer, khai báo trong file dự án Frogjump.touchgfx, gồm 3 màn hình (Screens).



Hình 9. Các màn hình của trò chơi (Màn hình chính, In-game, Kết thúc)

3.1.1. Screen1 - Màn hình khởi động (Start Screen)

- background: ảnh nền toàn màn hình 240×320 (start_screen.png)
- start_button: ảnh nút bấm bắt đầu, vùng chạm (12,240)-(132,320)
- Sự kiện chạm (RELEASED) trong vùng nút → chuyển sang Screen2 (gotoScreen2ScreenNoTransition)

3.1.2. Screen2 - Màn hình chơi chính (Gameplay)

Màn hình được thiết kế dàn theo 10 hàng ngang, mỗi hàng cao 32px, mô phỏng bản đồ Frogger. Cách bố trí này giúp phân chia các làn đường, khu vực sông và đích đến, đồng thời hỗ trợ quản lý vị trí, va chạm và di chuyển của nhân vật trong quá trình chơi.

Y (px)	Thành phần	Vai trò
288	grass1	Vùng xuất phát an toàn
256/224/192/160	road1..4 + các car_x_r/l	4 làn đường, xe di chuyển

128	grass2	Vùng an toàn giữa (đường & sông)
96/64/32	river1..3 + log_1/2, lotus_pad, crocodie	3 làn sông, khúc gỗ trôi
0	grass3	Vùng đích

Bảng 4: Bảng vị trí của các thành phần trên màn hình

- Nhân vật chính: cat (32×32), khởi đầu tại (104, 288)
- HUD: 3 heart (mạng sống), score (TextArea auto-size, wildcard hiển thị số điểm), cash (vật phẩm cộng điểm)
- Toàn bộ đối tượng xe/gỗ/vật thể đích trong Designer chỉ là vị trí mẫu tĩnh ban đầu; khi chạy thực tế, code C++ (Screen2View.cpp) sẽ ẩn các đối tượng do Designer đặt (hideDesignerCars(), hideDesignerRiverObjs()). Thay vào đó hệ thống sinh động lại toàn bộ chương ngại vật bằng các mảng widget tĩnh (touchgfx::Image carWidgets[...][...] và logWidgets[...]) để hỗ trợ tạo các hiệu ứng chuyển động và tái sử dụng bộ nhớ thay vì cấp phát hay hủy liên tục.

3.1.3. Screen3 - Màn hình kết thúc (End Screen)

- end_screen: ảnh nền
- scoreSummary, highestScoreSummary: hiển thị điểm hiện tại & điểm cao nhất
- 2 nút restart (vùng chạm 31,243-83,298 → quay lại Screen2) và home (vùng chạm 156,243-208,298 → quay lại Screen1)

3.1.4. Quản lý Text/Font

- texts.xml/texts.xsd: khai báo ngôn ngữ GB (English), 3 kiểu chữ (Typography) dùng font Verdana: Default (20px), Large (40px), Small (10px), Bpp=4 (16 mức xám antialias), có WildcardCharacters = "0123456789" để hiển thị số điểm động.

3.1.5. Chiến lược bộ đệm & màu sắc

- Color depth: 16-bit (RGB565)
- Bufferingstrategy: Doublebuffering (đặt trong STMICROELECTRONICS.X-CUBE-TOUCHGFX...tgfx_buffering_strategy=Double)

- Có hỗ trợ Vector Rendering (CWRVector) và Vector Fonts (theo AdditionalFeatures trong file cấu hình project), dù giao diện hiện tại chủ yếu dùng ảnh bitmap.

3.2. Kiến trúc mã nguồn, mô hình MVP

3.2.1. Kiến trúc phân tầng & Quản lý ứng dụng

a) FrontendApplication & FrontendHeap

- FrontendApplication kế thừa FrontendApplicationBase (sinh tự động), override handleTickEvent() để gọi model.tick() trước khi xử lý UI mỗi khung hình; cung cấp 3 hàm chuyển màn hình không hiệu ứng chuyển cảnh (NoTransition) thông qua callback (touchgfx::Callback) nhằm đảm bảo chuyển màn hình an toàn giữa các task.
- FrontendHeap cấp phát tĩnh (không dùng new/heap động) cho Presenter, View, Transition bằng touchgfx::Partition<..> - tránh phân mảnh bộ nhớ, phù hợp hệ nhúng tài nguyên hạn chế.

b) Lớp HAL/Target

- TouchGFXHAL/TouchGFXGeneratedHAL: khởi tạo framebuffer (đặt trong SDRAM qua section TouchGFX_Framebuffer), cấu hình ngắt LTDC/DMA2D, xử lý VSYNC.
- OSWrappers.cpp: cầu nối giữa framework TouchGFX và FreeRTOS (semaphore framebuffer, message queue VSYNC, osDelay, osThreadYield).
- STM32DMA.cpp: hiện thực DMA_Interface dùng phần cứng DMA2D.

3.2.2. Mô hình Model-View-Presenter

Phân hệ UI của trò chơi được phân chia thành 3 phần độc lập, giúp tách biệt giữa việc hiển thị, điều phối và dữ liệu.

a) Model (Model.hpp/.cpp)

- Lưu currentScore, highscore
- addScore(), resetCurrentScore(), notifyScore()
- tick() được gọi mỗi khung hình (từ FrontendApplication::handleTickEvent), đọc myCommandQueue (FreeRTOS) và gọi modelListener->onCommandReceived(cmd) - đây là cầu nối giữa lớp phần cứng/RTOS và lớp trình bày (MVP).

b) ModelListener (interface)

- bind(Model*), các hàm ảo onCommandReceived(), scoreUpdated() - mỗi Presenter kế thừa và override để nhận thông báo từ Model mà không phụ thuộc ngược lại View.

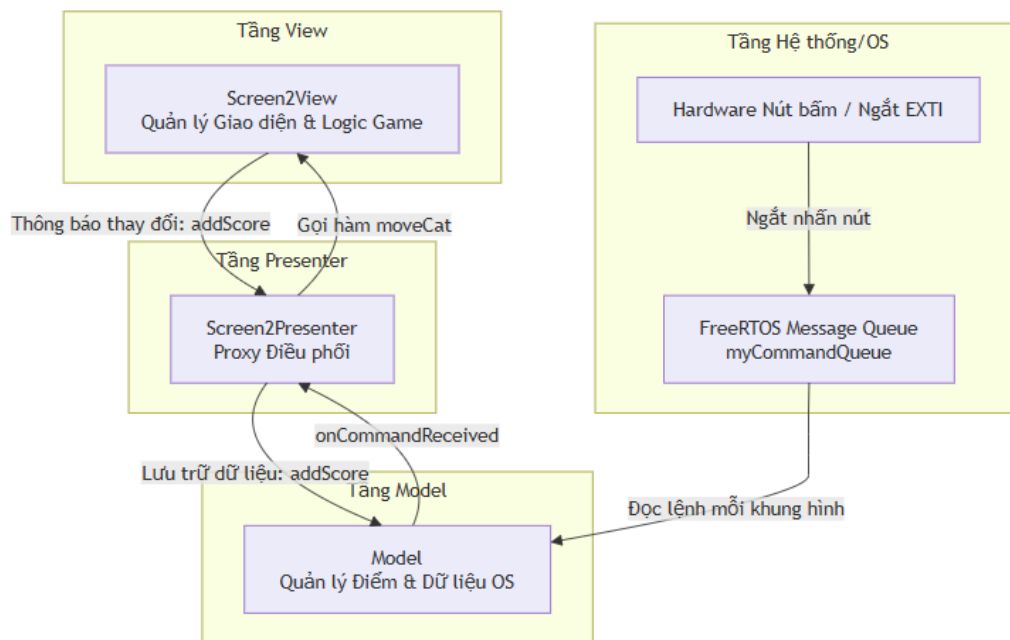
c) Presenter (3 lớp: Screen1/2/3Presenter)

Presenter	activate()	Vai trò riêng
Screen1Presenter	rỗng	Chỉ chuyển tiếp, không cần dữ liệu
Screen2Presenter	model->resetCurrentScore()	Chuyển tiếp onCommandReceived → view.moveCat(); addScore(), notifyScore() proxy tới Model
Screen3Presenter	model->notifyScore()	Nhận scoreUpdated() → view.updateScores()

Bảng 5. Bảng vai trò 3 lớp Presenter

d) View

- Screen1 View: xử lý chạm nút Start
- Screen2View: lớp phức tạp nhất, chứa toàn bộ logic game (xem mục 3.4)
- Screen3View: hiển thị điểm, xử lý nút Restart/Home



Hình 10. Sơ đồ hoạt động của game theo mô hình MVP

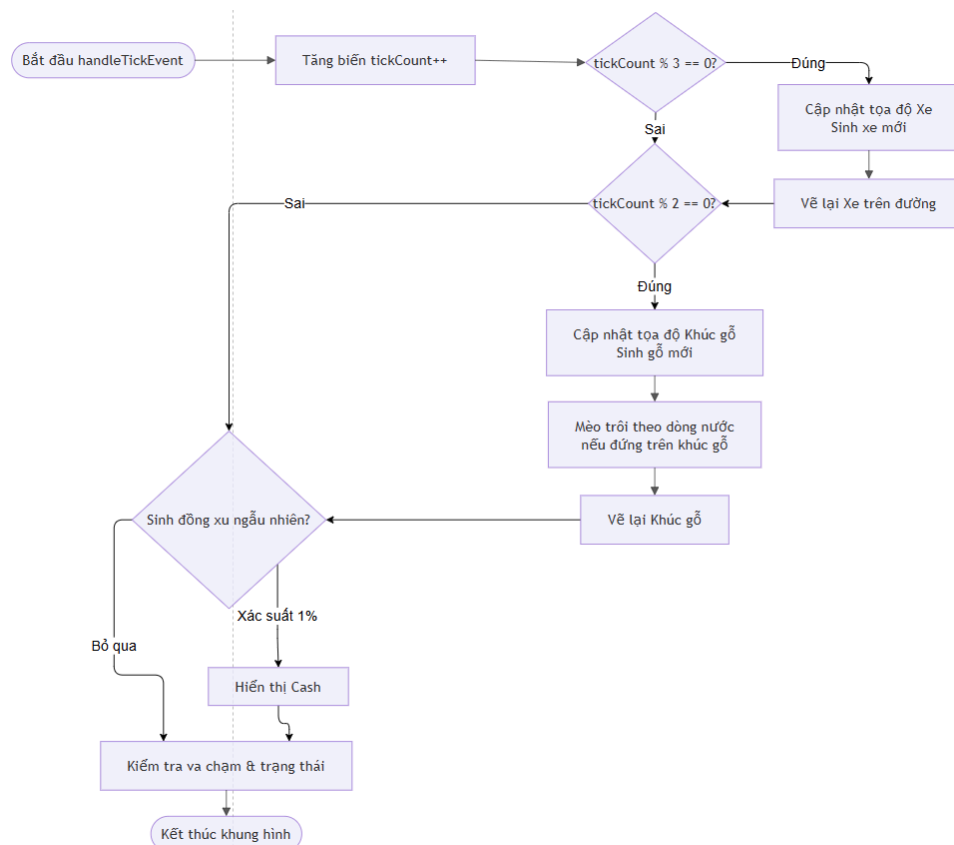
Mô hình MVP giúp tách bạch rõ ràng - Model không biết gì về giao diện, View không truy cập trực tiếp dữ liệu mà qua Presenter, điều này giúp thuận lợi hơn khi có nhu cầu mở rộng thêm màn hình hoặc thay đổi logic mà không ảnh hưởng tầng hiển thị.

3.3. Thuật toán Logic Game và xử lý sự kiện

3.3.1. Cấu trúc dữ liệu và vòng lặp game (handleTickEvent)

Mỗi làn xe (RoadLane) quản lý tối đa 3 xe (CarSlot). Mỗi làn sông (RiverLane) quản lý tối đa 3 khúc gỗ (LogSlot). Trạng thái game được cập nhật đồng bộ theo nhịp quét màn hình VSYNC, phân chia tần suất để cân bằng tải CPU:

- Cập nhật làn xe (đường bộ): Thực hiện mỗi 3 tick một lần (~20 FPS).
- Cập nhật làn gỗ (sông nước) và Mèo trôi theo sông: Thực hiện mỗi 2 tick một lần (~30 FPS). Nếu Mèo đang đứng trên làn sông nước, hệ thống kiểm tra xem tọa độ ngang của Mèo có nằm trong phạm vi nâng đỡ của khúc gỗ nào hay không. Nếu có, Mèo được tịnh tiến theo tốc độ và hướng trôi của khúc gỗ. Nếu Mèo trôi ra ngoài biên màn hình, người chơi mất mạng.
- Vật phẩm cash: Nếu chưa có đồng xu nào, mỗi tick game có xác suất ngẫu nhiên 1/100 để sinh một đồng xu trên làn đường nhựa ngẫu nhiên.



Hình 11. Sơ đồ luồng xử lý trong một chu kỳ (Tick) của trò chơi

3.3.2 Thuật toán sinh chương ngại vật giả ngẫu nhiên

Sử dụng thuật toán ngẫu nhiên XORShift32 thay thế hàm rand() thư viện chuẩn giúp tiết kiệm dung lượng Flash/RAM của MCU STM32:

```
uint32_t Screen2View::nextRandom()
{
    rngState ^= rngState << 13;
    rngState ^= rngState >> 17;
    rngState ^= rngState << 5;
    return rngState;
}
```

Seed ngẫu nhiên rngState được lấy từ HAL_GetTick(). Khi sinh xe hoặc gỗ mới ở rìa màn hình, thuật toán thực hiện kiểm tra khoảng cách an toàn với chương ngại vật gần nhất trên làn (MIN_CAR_GAP = 56px, MIN_LOG_GAP = 40px) để tránh hiện tượng chồng lấn hình ảnh. Chương ngại vật đi ra ngoài biên màn hình sẽ bị ẩn đi và tái sử dụng slot (Object Pooling).

3.3.3 Điều khiển di chuyển nhân vật và Xử lý va chạm

Quá trình điều khiển được thiết kế để Mèo di chuyển theo lưới ô. Bước nhảy dọc của Mèo bằng chiều cao làn (stepY = 32px), bước nhảy ngang bằng 60% bước dọc (stepX = 19px). Nhấn phím hợp lệ sẽ phát âm thanh SOUND_JUMP và gọi hàm check va chạm lập tức.

Hàm kiểm tra va chạm (checkCollisions) thực hiện kiểm tra va chạm bằng cách giới hạn phạm vi xét theo tọa độ Y hiện tại của Mèo.

- Đến Đích (Y = 0): Cộng cho người chơi 5 điểm, reset vị trí Mèo về xuất phát (104, 288).
- Trên làn đường: Kiểm tra xem tọa độ ngang của Mèo có giao với xe nào trên làn tương ứng không (biên dung sai margin = 4px). Nếu đụng xe, hệ thống sẽ phát âm thanh SOUND_CRASH và trừ 1 mạng. Nếu an toàn, hệ thống kiểm tra xem Mèo có ăn tiền vàng (cash) không để cộng 2 điểm, ẩn đồng tiền và phát âm thanh SOUND_CASH.
- Trên sông: Hệ thống kiểm tra xem Mèo có đứng trên khúc gỗ nào nâng đỡ không (biên dung sai nâng đỡ là 6px). Nếu không có khúc gỗ đỡ, Mèo rơi xuống nước, phát âm thanh SOUND_SINK_WATER và trừ 1 mạng.

Chương IV. Kết quả thực nghiệm & Đánh giá

4.1. Đánh giá độ trễ xử lý & Hiệu năng hệ thống

Hệ thống được thiết kế để xử lý đồng thời nhiều tác vụ nặng như render giao diện, tính toán logic game, phát âm thanh, nhưng thực nghiệm cho thấy trò chơi vẫn duy trì được độ mượt mà cao, không xảy ra hiện tượng xé hình (Tearing) hay nghẽn cổ chai (Bottleneck). Các kết quả đạt được nhờ vào chiến lược thiết kế sau:

- Tối ưu hóa Game Loop: Thay vì tính toán lại toàn bộ vị trí chương ngại vật ở mỗi khung hình (frame), thuật toán đã chia tải bằng cách sử dụng bộ đếm tickCount để xác định tần số sinh các vật thể. Đối với làn đường bộ, các đối tượng xe cộ được cập nhật mỗi 3 tick. Đối với làn sông, các vật thể khúc gỗ cập nhật mỗi 2 tick. Điều này giúp tránh việc render liên tục các vật thể mỗi một tick, từ đó giảm tải cho CPU.
- Đánh giá độ trễ Âm thanh (Audio Latency): Nhờ áp dụng phần cứng DMA vào hàm HAL_I2S_Transmit_DMA, quá trình truyền dữ liệu PCM từ Flash ra IC âm thanh không hề chặn luồng xử lý chính. Thực nghiệm khi nhân vật ăn liên tiếp vật phẩm (cash) hoặc rơi xuống nước, UI không hề bị khựng hay tụt FPS.
- Đánh giá phản hồi ngoại vi (nút bấm): Qua quá trình thử nghiệm thực tế, các nút bấm cho phản hồi tương đối tốt và nhạy, các thao tác từ người dùng được hệ thống ghi nhận gần như ngay lập tức và không phát hiện thấy độ trễ lớn gây ảnh hưởng đến trải nghiệm. Mặc dù hệ thống thu thập trạng thái nút bấm bằng phương pháp polling, tín hiệu nhận về vẫn đạt độ ổn định tốt. Ngoài ra, trong suốt quá trình vận hành thử nghiệm, hệ thống chưa ghi nhận hiện tượng nảy phím (key bouncing) hay hiện tượng nhận sai tín hiệu (loạn phím), đảm bảo các lệnh điều khiển được thực thi chính xác.

4.2. Đánh giá tài nguyên sử dụng bộ nhớ

Với đặc thù hệ thống nhúng STM32 có tài nguyên hạn chế, việc phân bổ bộ nhớ đã được đánh giá và quy hoạch chặt chẽ để hệ thống không bị tràn (HardFault/StackOverflow):

- Bộ nhớ External SDRAM (RAM ngoài): Được sử dụng tối đa cho tính năng Double Buffering (Cấp bộ đệm khung hình). Với độ phân giải 240×320 pixel và hệ màu 16-bit (RGB565), mỗi bộ đệm tốn xấp xỉ 153.6

KB. Tổng tài nguyên SDRAM sử dụng là 307.2 KB. Việc đặt Framebuffer ra RAM ngoài giải phóng toàn bộ áp lực cho RAM nội của vi điều khiển.

- Bộ nhớ Internal Flash (ROM): Chứa toàn bộ mã lệnh biên dịch, ảnh đồ họa tĩnh và font chữ. Đặc biệt, dữ liệu âm thanh thô (hàng chục ngàn phần tử uint16_t trong jump_sound.h, crash_sound.h) được đưa vào Flash để tránh gây tràn RAM. ROM đã hoàn thành tốt vai trò kho lưu trữ cục bộ tốc độ cao.
- Bộ nhớ Internal SRAM (RAM nội): Nhờ đẩy Framebuffer ra SDRAM, bộ nhớ SRAM quý giá bên trong MCU được dùng hoàn toàn cho Mảng quản lý hệ điều hành (Task Stacks, Message Queues của FreeRTOS) và Giao diện cấp phát tĩnh (FrontendHeap). Nhờ thiết kế tĩnh (Partitioning) không dùng con trỏ new/malloc, dự án đạt đánh giá độ an toàn bộ nhớ 100%, triệt tiêu hoàn toàn nguy cơ rò rỉ hay phân mảnh bộ nhớ sau thời gian dài người dùng chơi game.

Chương V. Kết luận & Hướng phát triển

5.1. Kết quả đạt được

- Xây dựng hoàn chỉnh một game 2D dạng Frogger ("Frogjump") chạy trên kit STM32F429I-Discovery, sử dụng framework đồ họa TouchGFX kết hợp FreeRTOS.
- Thiết kế đầy đủ 3 màn hình: Start - Gameplay - End, có chuyển màn hình, lưu điểm số & điểm cao nhất.
- Cài đặt thành công 3 hệ thống chương ngại vật độc lập (xe cộ, khúc gỗ trôi, vùng đích tĩnh) với sinh ngẫu nhiên có kiểm soát khoảng cách.
- Áp dụng đúng mô hình kiến trúc MVP chuẩn của TouchGFX, tách biệt rõ Model/View/Presenter.
- Tận dụng phần cứng tăng tốc đồ họa (DMA2D) và bộ nhớ SDRAM ngoài cho framebuffer.
- Thiết lập hạ tầng đo hiệu năng qua GPIO debug, phục vụ đánh giá và tối ưu về sau.

5.2. Những hạn chế

5.2.1. Lỗi tích hợp và xung đột trong quá trình sinh mã nguồn tự động

Một trong những vấn đề nhóm gặp phải nhiều nhất là quá trình sinh mã tự động từ STM32CubeMX và TouchGFX Designer. Mỗi khi thay đổi cấu hình phần cứng hoặc chỉnh sửa giao diện, việc thực hiện lệnh Generate Code thường làm ghi đè lên cấu hình của dự án trong IDE. Hậu quả là nhiều thư mục nguồn tự thêm như thư mục chứa âm thanh, các driver BSP tùy chỉnh hoặc các tệp cấu hình của project bị mất liên kết hoặc bị loại khỏi quá trình biên dịch. Điều này dẫn đến hàng loạt lỗi như không tìm thấy tệp tiêu đề, thiếu định nghĩa hàm hoặc lỗi liên kết khi biên dịch.

Từ thực tế đó, nhóm nhận thấy việc quản lý mã nguồn ngay từ đầu là hết sức cần thiết. Toàn bộ phần mã tự viết cần được đặt trong các vùng USER CODE do STM32CubeMX quy định nhằm tránh bị ghi đè khi sinh mã mới. Bên cạnh đó, việc sử dụng Git giúp theo dõi lịch sử thay đổi, nhanh chóng khôi phục các cấu hình bị mất cũng như so sánh sự khác biệt giữa các phiên bản sau mỗi lần sinh mã. Ngoài ra, thay vì thêm thủ công các thư mục nguồn trong IDE, việc khai báo đầy đủ đường dẫn Include và Source Folder ngay từ cấu hình dự án sẽ giúp giảm đáng kể các lỗi phát sinh trong quá trình phát triển.

Qua vấn đề này, nhóm nhận thấy rằng quản lý cấu trúc dự án và mã nguồn là yếu tố quan trọng không kém việc lập trình chức năng. Một hệ thống được tổ chức tốt sẽ giúp giảm thời gian sửa lỗi và nâng cao hiệu quả phát triển lâu dài.

5.2.2. Hiện tượng dội phím (bouncing) đối với các nút bấm

Hệ thống sử dụng bốn nút nhấn vật lý để điều khiển nhân vật trong trò chơi. Ban đầu, trạng thái các nút được đọc theo cơ chế Polling với chu kỳ khoảng 20 ms. Mặc dù cách tiếp cận này đơn giản và dễ triển khai, nhưng trong quá trình thử nghiệm nhóm nhận thấy hiện tượng rung tiếp điểm cơ học (contact bounce) khiến một lần nhấn vật lý có thể bị nhận thành nhiều lần nhấn liên tiếp. Điều này làm nhân vật đôi khi di chuyển nhiều bước ngoài mong muốn, ảnh hưởng trực tiếp đến trải nghiệm của người chơi.

Qua quá trình phân tích, nhóm nhận thấy cần bổ sung cơ chế chống dội phím. Một giải pháp hiệu quả là sử dụng Software Debouncing, trong đó trạng thái nút chỉ được xác nhận khi giữ ổn định trong một khoảng thời gian xác định trước khi gửi lệnh điều khiển vào hệ thống. Ngoài ra, nếu thiết kế phần cứng cho phép, có thể kết hợp thêm mạch RC để lọc nhiễu hoặc sử dụng ngắt ngoài (EXTI) cùng bộ định thời nhằm hạn chế các tín hiệu giả phát sinh do rung tiếp điểm.

Bài học rút ra là việc xử lý tín hiệu đầu vào tưởng chừng đơn giản nhưng lại có ảnh hưởng rất lớn đến chất lượng của toàn bộ hệ thống. Đối với các ứng dụng yêu cầu tương tác thời gian thực như trò chơi, việc xử lý tín hiệu chính xác và ổn định là điều kiện cần để mang lại trải nghiệm tốt cho người sử dụng.

5.2.3. Chưa tích hợp nhạc nền trò chơi

Hệ thống âm thanh của trò chơi hiện sử dụng giao tiếp I2S kết hợp với DMA để phát các hiệu ứng âm thanh như nhảy, thu thập vật phẩm và Game Over với độ trễ thấp. Tuy nhiên, trong quá trình phát triển, nhóm chưa tích hợp chức năng phát nhạc nền trong suốt quá trình chơi.

Nguyên nhân là nhạc nền yêu cầu dữ liệu âm thanh phải được truyền liên tục và ổn định trong suốt quá trình chơi, trong khi các hiệu ứng âm thanh lại phát sinh không đồng bộ theo thao tác của người dùng. Việc vừa duy trì phát nhạc nền liên tục, vừa đảm bảo các hiệu ứng âm thanh được phát đúng thời điểm là một bài toán tương đối phức tạp, đòi hỏi phải nghiên cứu thêm về cơ chế phát âm thanh trên hệ thống nhúng. Do giới hạn về thời gian thực hiện đề án, nhóm chưa có điều kiện nghiên cứu và triển khai nội dung này.

5.3. Hướng phát triển

Từ những hạn chế đã gặp trong quá trình thiết kế và triển khai, nhóm nhận thấy dự án vẫn còn nhiều hướng để tiếp tục hoàn thiện trong tương lai.

Trước hết, hệ thống âm thanh có thể được nâng cấp bằng cách xây dựng bộ trộn âm thanh (Software Audio Mixer) kết hợp với cơ chế DMA Circular Buffer, cho phép phát đồng thời nhạc nền và các hiệu ứng âm thanh mà không làm gián đoạn lẫn nhau. Đồng thời, việc lưu trữ các tệp âm thanh trên bộ nhớ ngoài hoặc sử dụng các định dạng nén phù hợp sẽ giúp tối ưu dung lượng bộ nhớ Flash của vi điều khiển.

Đối với hệ thống điều khiển, cơ chế đọc nút nhấn có thể được cải tiến bằng cách kết hợp thuật toán chống dội phím (Debounce) với ngắt ngoài (EXTI) hoặc bộ định thời (Timer), thay cho phương pháp Polling hiện tại. Giải pháp này sẽ nâng cao độ ổn định của tín hiệu đầu vào, giảm hiện tượng nhận nhiều lần nhấn ngoài ý muốn và cải thiện trải nghiệm điều khiển của người chơi.

Về mặt đồ họa, trò chơi có thể được tối ưu hơn bằng cách khai thác hiệu quả bộ tăng tốc Chrom-ART (DMA2D) của STM32F429 để giảm tải cho CPU, đồng thời thu hẹp vùng cập nhật màn hình và xây dựng cơ chế va chạm với Hitbox phù hợp hơn với hình dạng thực tế của các đối tượng. Những cải tiến này sẽ giúp tăng tốc độ hiển thị, giảm hiện tượng giật hình và nâng cao độ chính xác trong quá trình phát hiện va chạm.

Bên cạnh các cải tiến về kỹ thuật, dự án cũng có thể được mở rộng về mặt nội dung nhằm tăng tính hấp dẫn của trò chơi. Cụ thể, có thể bổ sung thêm nhiều màn chơi với độ khó tăng dần, đa dạng hóa các loại chướng ngại vật và vật phẩm, đồng thời xây dựng hệ thống tính điểm, lưu thành tích và lựa chọn mức độ khó. Những tính năng này không đòi hỏi thay đổi lớn về kiến trúc phần cứng nhưng sẽ giúp nâng cao trải nghiệm người dùng và tăng tính hoàn thiện của sản phẩm.

5.4 Lời cảm ơn

Nhóm xin chân thành cảm ơn thầy Ngô Lam Trung đã tận tình hướng dẫn, hỗ trợ và tạo điều kiện thuận lợi trong suốt quá trình học tập cũng như thực hiện đề tài. Những ý kiến đóng góp, sự chỉ bảo tận tâm và kinh nghiệm quý báu của thầy đã giúp nhóm hoàn thiện sản phẩm, đồng thời trau dồi thêm nhiều kiến thức và kỹ

năng thực tiễn, tạo nền tảng vững chắc cho các thành viên hướng tới thực hiện các dự án trong tương lai.

Trong quá trình thực hiện đề tài, mặc dù nhóm đã cố gắng nghiên cứu, tìm hiểu và triển khai một cách nghiêm túc, nhưng do thời gian thực hiện cùng kiến thức và kinh nghiệm còn hạn chế, đề tài vẫn không tránh khỏi những thiếu sót nhất định. Nhóm kính mong các thầy/cô thông cảm cũng như đưa ra thêm các góp ý để nhóm khắc phục được những thiếu sót còn tồn đọng, hoàn thiện bản thân mình hơn.

PHỤ LỤC

Mã nguồn toàn bộ đề tài: *<https://github.com/3XChehe/Frog-jump>*